Докладчик

- Жаворонков Кирилл Александрович
- Студент
- Группа: НПИбд-01-23
- Номер студенческого билета: 1132231844

Цель работы

- Изучить возможности пакетов Julia для операций линейной алгебры.
- Оценить эффективность операций над матрицами.
- Выполнить примеры раздела 4.2 и задания 4.4.1–4.4.4.

Слайд 1. Суммы, произведения, средние

- Поэлементные операции по всей матрице, по столбцам и по строкам.
- sum(a) = 159; mean(a) = 13.25.

```
[1]: a = rand(1:20, (4,3))

[1]: 4×3 Matrix{Int64}:
      13   5  17
       2  10  19
      15  20  11
       7  20  20

[2]: sum(a)

[2]: 159

[3]: sum(a, dims=1)

[3]: 1×3 Matrix{Int64}:
      37  55  67

[4]: sum(a, dims=2)

[4]: 4×1 Matrix{Int64}:
      35
      31
      46
      47

[5]: prod(a)

[5]: 3879876000000

[6]: prod(a, dims=1)

[6]: 1×3 Matrix{Int64}:
      2730  20000  71060
```

Слайд 2. Произведения и статистика

- prod с параметром dims и среднее значение пакета Statistics.

```
2750  20000  71000

[7]: prod(a, dims=2)

[7]: 4×1 Matrix{Int64}:
      1105
       380
      3300
      2800

[8]: import Pkg
     Pkg.add("Statistics")

     Updating registry at `C:\Users\Кирилл\.julia\registries\General.toml`
    Resolving package versions...
     Updating `C:\Users\Кирилл\.julia\environments\v1.12\Project.toml`
  [10745b16] + Statistics v1.11.5
     Manifest No packages added to or removed from `C:\Users\Кирилл\.julia\environments\v
1.12\Manifest.toml`

[10]: using Statistics

[11]: mean(a)

[11]: 13.25

[12]: mean(a, dims=1)

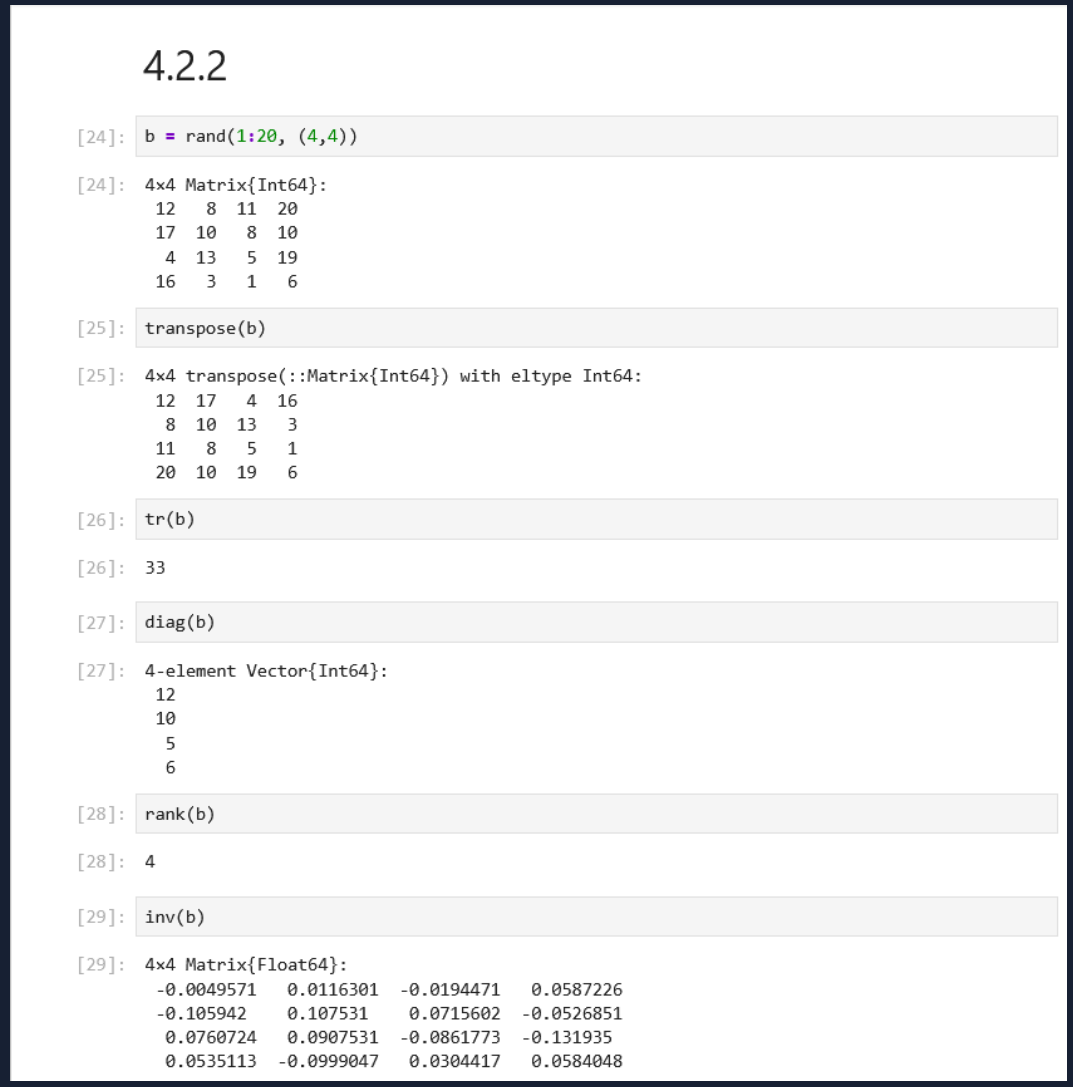
[12]: 1×3 Matrix{Float64}:
       9.25  13.75  16.75

[13]: mean(a, dims=2)

[13]: 4×1 Matrix{Float64}:
       11.666666666666666
       10.333333333333334
       15.333333333333334
       15.666666666666666
```

Слайд 3. Транспонирование, след, ранг, инверсия

- tr(b) = 33; diag(b) = [12, 10, 5, 6]; rank(b) = 4.
- inv(b) и det(b) = 15735.0.



4.2.2

```
[24]: b = rand(1:20, (4,4))
```

```
[24]: 4×4 Matrix{Int64}:
       12   8  11  20
       17  10   8  10
        4  13   5  19
       16   3   1   6
```

```
[25]: transpose(b)
```

```
[25]: 4×4 transpose(::Matrix{Int64}) with eltype Int64:
       12  17   4  16
        8  10  13   3
       11   8   5   1
       20  10  19   6
```

```
[26]: tr(b)
```

```
[26]: 33
```

```
[27]: diag(b)
```

```
[27]: 4-element Vector{Int64}:
       12
       10
        5
        6
```

```
[28]: rank(b)
```

```
[28]: 4
```

```
[29]: inv(b)
```

```
[29]: 4×4 Matrix{Float64}:
       -0.0049571   0.0116301  -0.0194471   0.0587226
       -0.105942    0.107531    0.0715602  -0.0526851
        0.0760724   0.0907531  -0.0861773  -0.131935
        0.0535113  -0.0999047   0.0304417   0.0584048
```

Слайд 4. Определитель и псевдообратная матрица

- det(b) = 15735.0.
- pinv(b) вычислена для той же матрицы.

```
 0.0535113  -0.0999047   0.0304417   0.0584048

[30]: det(b)

[30]: 15735.0

[31]: pinv(b)

[31]: 4×4 Matrix{Float64}:
 -0.0049571   0.0116301  -0.0194471   0.0587226
 -0.105942    0.107531    0.0715602  -0.0526851
  0.0760724   0.0907531  -0.0861773  -0.131935
  0.0535113  -0.0999047   0.0304417   0.0584048
```

Слайд 5. Нормы и расстояние

- norm(X, 1) = 11.0; расстояние ||X − Y|| ≈ 9.4868.
- Угол между векторами ≈ 2.4404 рад.



4.2.3

```
[32]: X = [2, 4, -5]
```

```
[32]: 3-element Vector{Int64}:
        2
        4
       -5
```

```
[33]: Y = [1, -1, 3]
```

```
[33]: 3-element Vector{Int64}:
        1
       -1
        3
```

```
[34]: p = 1
```

```
[34]: 1
```

```
[35]: norm(X, p)
```

```
[35]: 11.0
```

```
[36]: norm(X-Y)
```

```
[36]: 9.486832980505138
```

```
[38]: sqrt(sum((X-Y).^2))
```

```
[38]: 9.486832980505138
```

```
[39]: acos((transpose(X)*Y) / (norm(X)*norm(Y)))
```

```
[39]: 2.4404307889469252
```

Слайд 6. Нормы матриц и повороты

- opnorm(d) ≈ 7.0458; opnorm(d, 1) = 8.0.
- rot180 и reverse по строкам и столбцам.

```
[40]: d = [5 -4 2; 1 2 3; -2 1 0]

[40]: 3×3 Matrix{Int64}:
        5  -4  2
        1   2  3
       -2   1  0

[41]: opnorm(d)

[41]: 7.0457718132748814

[42]: opnorm(d, 1)

[42]: 8.0

[44]: rot180(d)

[44]: 3×3 Matrix{Int64}:
       0   1  -2
       3   2   1
       2  -4   5

[45]: reverse(d, dims=1)

[45]: 3×3 Matrix{Int64}:
       -2   1  0
        1   2  3
        5  -4  2

[46]: reverse(d, dims=2)

[46]: 3×3 Matrix{Int64}:
       2  -4   5
       3   2   1
       0   1  -2
```

Слайд 7. Матричное умножение

- Произведение матриц A и B; единичная матрица Matrix{Int}(I, 3, 3)
- dot(X, Y) = −17.

4.2.4

```
[50]: A = rand(1:10, (2,3))

[50]: 2×3 Matrix{Int64}:
        6   5  4
       10  10  9

[51]: B = rand(1:10, (3,4))

[51]: 3×4 Matrix{Int64}:
       5  6  2  2
       2  7  7  7
       9  9  9  2

[52]: A * B

[52]: 2×4 Matrix{Int64}:
        76  107   83   55
       151  211  171  108

[53]: Matrix{Int}(I,3,3)

[53]: 3×3 Matrix{Int64}:
       1  0  0
       0  1  0
       0  0  1

[54]: dot(X, Y)

[54]: -17
```

Слайд 8. Решение СЛАУ и LU-разложение

- Решение A·x = b через оператор \ и LU-разложение.
- det(A) = det(Alu) ≈ 0.02694.

4.2.5

[55]:
```
A_sq = rand(3, 3)
```

[55]:
```
3×3 Matrix{Float64}:
 0.13608    0.426096  0.565069
 0.273626   0.506927  0.186691
 0.0120762  0.371511  0.390056
```

[56]:
```
x_vec = fill(1.0, 3)
```

[56]:
```
3-element Vector{Float64}:
 1.0
 1.0
 1.0
```

[57]:
```
b_vec = A_sq * x_vec
```

[57]:
```
3-element Vector{Float64}:
 1.1272450820390736
 0.9672447054636377
 0.7736429046682447
```

[58]:
```
A_sq \ b_vec
```

[58]:
```
3-element Vector{Float64}:
 0.9999999999999996
 1.0000000000000002
 0.9999999999999998
```

[59]:
```
Alu = lu(A_sq)
```

[59]:
```
LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0        0.0       0.0
 0.0441341  1.0       0.0
 0.49732    0.498345  1.0
U factor:
3×3 Matrix{Float64}:
 0.273626  0.506927  0.186691
 0.0       0.349138  0.381816
 0.0       0.0       0.281947
```

Слайд 9. Факторы LU-разложения

- Матрицы P, p, L и U из объекта Alu.

```
[60]: Alu.P

[60]: 3×3 Matrix{Float64}:
       0.0  1.0  0.0
       0.0  0.0  1.0
       1.0  0.0  0.0

[61]: Alu.p

[61]: 3-element Vector{Int64}:
       2
       3
       1

[62]: Alu.L

[62]: 3×3 Matrix{Float64}:
       1.0        0.0       0.0
       0.0441341  1.0       0.0
       0.49732    0.498345  1.0

[63]: Alu.U

[63]: 3×3 Matrix{Float64}:
       0.273626  0.506927  0.186691
       0.0       0.349138  0.381816
       0.0       0.0       0.281947

[64]: Alu \ b_vec

[64]: 3-element Vector{Float64}:
       0.9999999999999996
       1.0000000000000002
       0.9999999999999998

[65]: det(A_sq)

[65]: 0.026935400788848776

[66]: det(Alu)

[66]: 0.026935400788848776
```

Слайд 10. QR-разложение и симметризация

- Матрицы Q и R; Q′Q приближённо равна единичной матрице.
- Asym = A + A^T.

```
[65]: det(A_sq)

[65]: 0.026935400788848776

[66]: det(Alu)

[66]: 0.026935400788848776

[67]: Aqr =qr(A_sq)

[67]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
      Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
      R factor:
      3×3 Matrix{Float64}:
       -0.305835  -0.6578    -0.433857
        0.0        0.379279   0.513953
        0.0        0.0       -0.232208

[68]: Aqr.Q

[68]: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}

[69]: Aqr.R

[69]: 3×3 Matrix{Float64}:
       -0.305835  -0.6578    -0.433857
        0.0        0.379279   0.513953
        0.0        0.0       -0.232208

[70]: Aqr.Q' * Aqr.Q

[70]: 3×3 Matrix{Float64}:
         1.0          5.55112e-17  -2.22045e-16
        -1.11022e-16  1.0           1.11022e-16
        -9.02056e-17  0.0           1.0

[71]: Asym = A_sq + A_sq'

[71]: 3×3 Matrix{Float64}:
       0.27216   0.699723  0.577145
       0.699723  1.01385   0.558202
       0.577145  0.558202  0.780112
```

Слайд 11. Спектральное разложение

- Собственные значения: −0.2165; 0.3267; 1.9559.

```
[72]: AsymEig = eigen(Asym)

[72]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
3-element Vector{Float64}:
 -0.21650239103715974
  0.3267368765003318
  1.955891251385182
vectors:
3×3 Matrix{Float64}:
  0.880341   0.00875506  -0.47426
 -0.361136  -0.63587     -0.682093
 -0.307539   0.771747    -0.55662

[73]: AsymEig.values

[73]: 3-element Vector{Float64}:
 -0.21650239103715974
  0.3267368765003318
  1.955891251385182

[74]: AsymEig.vectors

[74]: 3×3 Matrix{Float64}:
  0.880341   0.00875506  -0.47426
 -0.361136  -0.63587     -0.682093
 -0.307539   0.771747    -0.55662

[75]: inv(AsymEig.vectors) * Asym

[75]: 3×3 Matrix{Float64}:
 -0.190596    0.0781867   0.066583
  0.0028606  -0.207762    0.252158
 -0.927601   -1.3341     -1.08869
```

Слайд 12. Матрица 1000×1000

- issymmetric(Asym) = true.
- Добавление шума 5·eps() даёт issymmetric = false.

```
[77]: n = 1000

[77]: 1000

[78]: A_large = randn(n, n)

[78]: 1000×1000 Matrix{Float64}:
 -0.615858   -0.228031     -0.305955   …   1.30499     1.23747    -0.209646
 -0.937328   -0.514224      0.959626       0.531668   -1.95033     0.30733
 -0.501953   -0.81049      -0.723275      -1.20472     1.44252     1.07446
  0.925      -0.887481      0.995631       0.157183   -0.94262     0.0529531
 -0.830315    0.418896     -2.48229        0.109057    0.0214578  -0.962279
  0.253901   -0.0111574     0.505954   …   1.03733     1.07919     1.34041
 -0.147267    1.67563      -0.0956893      0.0460686   1.15731    -0.0803317
 -0.536698    1.73037       1.64574       -0.911087   -0.252225   -0.0394915
  0.258152    0.33043       0.848749      -0.13741    -1.23111     0.682769
 -2.65985    -0.768362      1.89153       -0.27905    -1.4453     -1.97614
 -1.34558    -1.28562      -0.982734   …  -0.557113   -0.393067   -0.312067
  1.52298     1.04159       0.319376       1.52446     1.08856     2.1253
  0.393872   -0.000685162  -0.155605       0.447162   -0.412265   -0.663951
  ⋮                                    ⋱
 -0.463358    2.2678        0.315588       0.135262    0.418058   -0.616542
 -0.801599    0.805446      0.410775       0.399634    2.30938    -0.141757
  0.402956    0.95164       0.973347   …  -0.0886386   0.863799    0.352805
  0.748845   -1.6609       -1.17153       -0.298516    0.0186388  -2.90324
  0.041425   -0.810487      0.496367       0.692612    0.127588    1.37191
 -0.190743   -1.04254      -0.0184249     -0.158114   -0.531231    0.557482
 -0.333816   -0.0608881    -1.61407       -1.09012    -1.05643     0.14544
 -0.0390414   0.700894     -1.36135   …   -1.69741    -0.697051   -0.60601
 -0.716964   -0.248236      0.888765      -1.05566     0.802012    0.397862
 -0.694593    0.642775     -0.933368      -0.722201   -0.308115    1.21125
  0.805186   -0.738262      0.268333      -0.275475   -0.739259    0.269236
  0.621176    1.98414      -0.611896      -1.98232     0.0823949  -0.325224
```

Слайд 13. Шум и тип Symmetric

- Явное указание Symmetric(Asym_noisy).
- Установлен пакет BenchmarkTools.

```
 0.621176    1.98414     -0.611896     -1.98232     0.0823949  -0.325224

[79]: Asym_large = A_large + A_large'

[79]: 1000×1000 Matrix{Float64}:
 -1.23172     -1.16536   -0.807908   0.698533  …   2.04265     0.41153
 -1.16536     -1.02845    0.149136   0.22447      -2.6886      2.29147
 -0.807908     0.149136  -1.44655   -0.108882      1.71085     0.462568
  0.698533     0.22447   -0.108882  -0.782682      0.475884    0.336337
 -0.787615     2.01037   -2.77435   -3.05985       0.232674   -1.42231
  0.206914     2.34672   -0.648236  -1.34822   …   0.648299   -0.287121
  0.684938     0.276373   1.25692   -2.33291       0.226669   -0.344371
 -1.02509      0.801111   1.80596   -0.628925     -0.120396    0.162304
 -1.18566      0.247933   0.552203   0.876248      0.0250057  -0.076451
 -2.93673     -1.13487    2.18146    0.603258     -2.67745    -1.9026
 -0.00473393  -1.03807    0.303172   0.726954  …  -0.0118973   0.333831
  1.98644     -0.716931   0.446868   0.483578      0.806555    2.45732
  2.34487      1.01776   -0.248203  -1.06982      -0.107497   -2.95961
  ⋮                                                ⋱
 -0.244352    2.24243    0.398307   2.07201       0.761257   -0.253152
  0.172108    2.61176   -0.776744   0.631911      1.66494     0.0346976
 -0.230751    0.573075   1.03056   -1.46231   …  -0.655206   -0.245226
  0.667446    0.169016  -1.90019   -0.825707      0.213425   -1.67781
  1.15474     0.409613  -0.38593   -2.44237      -1.66042     0.86389
 -0.604315   -1.80743   -0.149082   0.306361      0.365298   -2.32122
  0.944017    1.17112   -3.74005    1.57394      -0.0151415   1.12964
  0.839751    1.83262   -1.46633    0.878743  …  -1.47603    -0.224787
 -1.33456     1.31828    0.149575   1.34084      -0.492457    2.08401
  0.610395    1.17444   -2.13809    0.693209     -0.58359    -0.77107
  2.04265    -2.6886     1.71085    0.475884     -1.47852     0.351631
  0.41153     2.29147    0.462568   0.336337      0.351631   -0.650448

[80]: issymmetric(Asym_large)

[80]: true
```

Слайд 14. Типы матриц и время вычисления

- eigvals для Symmetric: 56.033 ms; для зашумлённой матрицы: 366.536 ms.

```
[81]: Asym_noisy = copy(Asym_large)
```

```
[81]: 1000×1000 Matrix{Float64}:
 -1.23172     -1.16536   -0.807908   0.698533  …   2.04265     0.41153
 -1.16536     -1.02845    0.149136   0.22447      -2.6886      2.29147
 -0.807908     0.149136  -1.44655   -0.108882      1.71085     0.462568
  0.698533     0.22447   -0.108882  -0.782682      0.475884    0.336337
 -0.787615     2.01037   -2.77435   -3.05985       0.232674   -1.42231
  0.206914     2.34672   -0.648236  -1.34822   …   0.648299   -0.287121
  0.684938     0.276373   1.25692   -2.33291       0.226669   -0.344371
 -1.02509      0.801111   1.80596   -0.628925     -0.120396    0.162304
 -1.18566      0.247933   0.552203   0.876248      0.0250057  -0.076451
 -2.93673     -1.13487    2.18146    0.603258     -2.67745    -1.9026
 -0.00473393  -1.03807    0.303172   0.726954  …  -0.0118973   0.333831
  1.98644     -0.716931   0.446868   0.483578      0.806555    2.45732
  2.34487      1.01776   -0.248203  -1.06982      -0.107497   -2.95961
  ⋮                                              ⋱
 -0.244352     2.24243    0.398307   2.07201       0.761257   -0.253152
  0.172108     2.61176   -0.776744   0.631911      1.66494     0.0346976
 -0.230751     0.573075   1.03056   -1.46231   …  -0.655206   -0.245226
  0.667446     0.169016  -1.90019   -0.825707      0.213425   -1.67781
  1.15474      0.409613  -0.38593   -2.44237      -1.66042     0.86389
 -0.604315    -1.80743   -0.149082   0.306361      0.365298   -2.32122
  0.944017     1.17112   -3.74005    1.57394      -0.0151415   1.12964
  0.839751     1.83262   -1.46633    0.878743  …  -1.47603    -0.224787
 -1.33456      1.31828    0.149575   1.34084      -0.492457    2.08401
  0.610395     1.17444   -2.13809    0.693209     -0.58359    -0.77107
  2.04265     -2.6886     1.71085    0.475884     -1.47852     0.351631
  0.41153      2.29147    0.462568   0.336337      0.351631   -0.650448
```

```
[82]: Asym_noisy[1,2] += 5eps()
```

```
[82]: -1.1653588349167083
```

```
[83]: issymmetric(Asym_noisy)
```

```
[83]: false
```

Слайд 15. Оценка времени

- Сравнение времени вычисления собственных значений.

```
[84]: Asym_explicit = Symmetric(Asym_noisy)
```

```
[84]: 1000×1000 Symmetric{Float64, Matrix{Float64}}:
 -1.23172     -1.16536    -0.807908    0.698533  …    2.04265     0.41153
 -1.16536     -1.02845     0.149136    0.22447      -2.6886      2.29147
 -0.807908     0.149136   -1.44655    -0.108882      1.71085     0.462568
  0.698533     0.22447    -0.108882   -0.782682      0.475884    0.336337
 -0.787615     2.01037    -2.77435    -3.05985       0.232674   -1.42231
  0.206914     2.34672    -0.648236   -1.34822   …   0.648299   -0.287121
  0.684938     0.276373    1.25692    -2.33291       0.226669   -0.344371
 -1.02509      0.801111    1.80596    -0.628925     -0.120396    0.162304
 -1.18566      0.247933    0.552203    0.876248      0.0250057  -0.076451
 -2.93673     -1.13487     2.18146     0.603258     -2.67745    -1.9026
 -0.00473393  -1.03807     0.303172    0.726954  …  -0.0118973   0.333831
  1.98644     -0.716931    0.446868    0.483578      0.806555    2.45732
  2.34487      1.01776    -0.248203   -1.06982      -0.107497   -2.95961
  ⋮                                                ⋱
 -0.244352     2.24243     0.398307    2.07201       0.761257   -0.253152
  0.172108     2.61176    -0.776744    0.631911      1.66494     0.0346976
 -0.230751     0.573075    1.03056    -1.46231   …  -0.655206   -0.245226
  0.667446     0.169016   -1.90019    -0.825707      0.213425   -1.67781
  1.15474      0.409613   -0.38593    -2.44237      -1.66042     0.86389
 -0.604315    -1.80743    -0.149082    0.306361      0.365298   -2.32122
  0.944017     1.17112    -3.74005     1.57394      -0.0151415   1.12964
  0.839751     1.83262    -1.46633     0.878743  …  -1.47603    -0.224787
 -1.33456      1.31828     0.149575    1.34084      -0.492457    2.08401
  0.610395     1.17444    -2.13809     0.693209     -0.58359    -0.77107
  2.04265     -2.6886      1.71085     0.475884     -1.47852     0.351631
  0.41153      2.29147     0.462568    0.336337      0.351631   -0.650448
```

```
[85]: Pkg.add("BenchmarkTools")
```

Слайд 16. Трёхдиагональная матрица 10^6

- SymTridiagonal: eigmax = 471.127 ms, 183.11 MiB.

```
        57.779 ms (18 allocations: 7.98 MiB)

[92]: @btime eigvals($Asym_noisy);
        366.536 ms (27 allocations: 7.93 MiB)

[93]: @btime eigvals($Asym_explicit);
        56.033 ms (18 allocations: 7.98 MiB)

[94]: n_tri = 1000000
      A_tri = SymTridiagonal(rand(n_tri), rand(n_tri-1))

[94]: 1000000×1000000 SymTridiagonal{Float64, Vector{Float64}}:
       0.178593   0.0470815   ·          ·          …   ·          ·          ·
       0.0470815  0.162308    0.588096   ·              ·          ·          ·
        ·         0.588096    0.518759   0.630455       ·          ·          ·
        ·          ·          0.630455   0.460618       ·          ·          ·
        ·          ·           ·         0.577487       ·          ·          ·
        ·          ·           ·          ·         …   ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·         …   ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ⋮                                                ⋱
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·         …   ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·             ·          ·          ·
        ·          ·           ·          ·         …   ·          ·          ·
        ·          ·           ·          ·             0.720705   ·          ·
        ·          ·           ·          ·             0.729164   0.281017   ·
        ·          ·           ·          ·             0.281017   0.871481   0.514717
        ·          ·           ·          ·              ·         0.514717   0.543953

[95]: @btime eigmax($A_tri);
        471.127 ms (38 allocations: 183.11 MiB)
```

Слайд 17. Рациональные матрицы

- Точное решение системы через BigInt: x = [1, 1, 1].
- LU-разложение рациональной матрицы.

4.2.6

```
[96]: Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3)) // 10
```

```
[96]: 3×3 Matrix{Rational{BigInt}}:
 4//5   1     3//10
  1    9//10  3//10
 4//5  1//2   1//5
```

```
[97]: x_rat = fill(1, 3)
```

```
[97]: 3-element Vector{Int64}:
 1
 1
 1
```

```
[98]: b_rat = Arational * x_rat
```

```
[98]: 3-element Vector{Rational{BigInt}}:
 21//10
 11//5
  3//2
```

```
[99]: Arational \ b_rat
```

```
[99]: 3-element Vector{Rational{BigInt}}:
 1
 1
 1
```

```
[101]: lu(Arational)
```

```
[101]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
L factor:
3×3 Matrix{Rational{BigInt}}:
  1       0     0
 4//5     1     0
 4//5  -11//14  1
U factor:
3×3 Matrix{Rational{BigInt}}:
 1  9//10  3//10
 0  7//25  3//50
 0   0     1//140
```

Слайд 18. Задания 4.4.1 и 4.4.2

- dot(v, v) = 29; внешнее произведение $v \cdot v^T$.
- Системы с двумя и тремя неизвестными через \ и pinv.

САМОСТОЯТЕЛЬНАЯ РАБОТА

```
[102]: v = [2, 3, 4]
```

```
[102]: 3-element Vector{Int64}:
        2
        3
        4
```

```
[103]: dot_v = dot(v, v)
```

```
[103]: 29
```

```
[104]: outer_v = v * v'
```

```
[104]: 3×3 Matrix{Int64}:
        4   6   8
        6   9  12
        8  12  16
```

```
[105]: x1_a = [1 1; 1 -1] \ [2, 3]
       x1_b = pinv([1 1; 2 2]) * [2, 4]
       x1_c = pinv([1 1; 2 2]) * [2, 5]
       x1_d = [1 1; 2 2; 3 3] \ [1, 2, 3]
       x1_e = [1 1; 2 1; 1 -1] \ [2, 1, 3]
       x1_f = [1 1; 2 1; 3 2] \ [2, 1, 3]
```

```
[105]: 2-element Vector{Float64}:
        -0.9999999999999994
         2.999999999999999
```

```
[106]: x2_a = pinv([1 1 1; 1 -1 -2]) * [2, 3]
       x2_b = [1 1 1; 2 2 -3; 3 1 1] \ [2, 4, 1]
       x2_c = pinv([1 1 1; 1 1 2; 2 2 3]) * [1, 0, 1]
       x2_d = pinv([1 1 1; 1 1 2; 2 2 3]) * [1, 0, 0]
```

```
[106]: 3-element Vector{Float64}:
         0.8333333333333324
         0.8333333333333346
        -1.0000000000000002
```

Слайд 19. Задание 4.4.3. Диагонализация и степени

- Диагональные матрицы из собственных значений.
- [1 −2; −2 1]^10 = [29525 −29524; −29524 29525].

```
[111]: diag_a = Diagonal(eigen([1 -2; -2 1]).values)

[111]: 2×2 Diagonal{Float64, Vector{Float64}}:
        -1.0    ·
         ·     3.0

[112]: diag_b = Diagonal(eigen([1 -2; -2 3]).values)

[112]: 2×2 Diagonal{Float64, Vector{Float64}}:
        -0.236068    ·
         ·          4.23607

[113]: diag_c = Diagonal(eigen([1 -2 0; -2 1 2; 0 2 0]).values)

[113]: 3×3 Diagonal{Float64, Vector{Float64}}:
        -2.14134    ·          ·
         ·         0.515138    ·
         ·          ·         3.6262

[116]: pow_a = [1 -2;-2 1]^10

[116]: 2×2 Matrix{Int64}:
         29525  -29524
        -29524   29525

[117]: sqrt_b = sqrt([5 -2; -2 5])

[117]: 2×2 Matrix{Float64}:
         2.1889   -0.45685
        -0.45685   2.1889

[118]: cbrt_c = Complex.([1 -2; -2 1])^(1/3)

[118]: 2×2 Matrix{ComplexF64}:
         0.971125+0.433013im  -0.471125+0.433013im
        -0.471125+0.433013im   0.971125+0.433013im

[119]: sqrt_d = sqrt([1 2; 2 3])

[119]: 2×2 Matrix{ComplexF64}:
        0.568864+0.351578im  0.920442-0.217287im
        0.920442-0.217287im   1.48931+0.134291im
```

Слайд 20. Собственные значения 5×5

- Собственные значения: −128.49; −55.89; 42.75; 87.16; 542.47.
- Диагональная и нижнетреугольная матрицы.

```
[120]: A_big = [140 97 74 168 131;
                97 106 89 131 36;
                74 89 152 144 71;
                168 131 144 54 142;
                131 36 71 142 36]

[120]: 5×5 Matrix{Int64}:
        140   97   74  168  131
         97  106   89  131   36
         74   89  152  144   71
        168  131  144   54  142
        131   36   71  142   36

[121]: eig_A = eigvals(A_big)

[121]: 5-element Vector{Float64}:
       -128.49322764802136
        -55.88778455305686
         42.75216727931889
         87.16111477514512
        542.4677301466143

[122]: A_diag = Diagonal(eig_A)

[122]: 5×5 Diagonal{Float64, Vector{Float64}}:
       -128.493      ⋅         ⋅         ⋅         ⋅
           ⋅     -55.8878      ⋅         ⋅         ⋅
           ⋅         ⋅      42.7522      ⋅         ⋅
           ⋅         ⋅         ⋅      87.1611      ⋅
           ⋅         ⋅         ⋅         ⋅      542.468

[123]: A_lower = LowerTriangular(A_big)

[123]: 5×5 LowerTriangular{Int64, Matrix{Int64}}:
        140    ⋅    ⋅    ⋅   ⋅
         97  106    ⋅    ⋅   ⋅
         74   89  152    ⋅   ⋅
        168  131  144   54   ⋅
        131   36   71  142  36

[124]: @btime eigvals($A_big);

         2.411 μs (13 allocations: 2.44 KiB)
```

Слайд 21. Задание 4.4.4. Функции продуктивности

- Три функции: по определению, по критерию $(E - A)^{-1} \geq 0$, спектральный

[125]:
```
function is_productive_def(A)
    I_mat = Matrix{Float64}(I, size(A,1), size(A,2))
    return all((I_mat - A) \ ones(size(A,1)) .>= 0)
end

function is_productive_crit(A)
    return all(inv(Matrix{Float64}(I, size(A,1), size(A,2)) - A) .>= 0)
end

function is_productive_spectral(A)
    return all(abs.(eigvals(A)) .< 1)
end
```

[125]:
```
is_productive_spectral (generic function with 1 method)
```

Слайд 22. Результаты проверки продуктивности

- (1 2; 3 1)/10 продуктивна по критериям $(E - A)^{-1}$ и спектральному.
- Матрицы (1 2; 3 1) и (1 2; 3 1)/2 не продуктивны.

```
[125]: is_productive_spectral (generic function with 1 method)

[126]: prod_def_a = is_productive_def([1 2; 3 4])

[126]: false

[127]: prod_def_b = is_productive_def([1 2; 3 4])

[127]: false

[128]: prod_def_c = is_productive_def((1/10) * [1 2; 3 4])

[128]: true

[129]: prod_crit_a = is_productive_crit([1 2; 3 1])

[129]: false

[130]: prod_crit_b = is_productive_crit((1/2) * [1 2; 3 1])

[130]: false

[131]: prod_crit_c = is_productive_crit((1/10) * [1 2; 3 1])

[131]: true

[132]: prod_spec_a = is_productive_spectral([1 2; 3 1])

[132]: false

[133]: prod_spec_b = is_productive_spectral((1/2) * [1 2; 3 1])

[133]: false

[134]: prod_spec_c = is_productive_spectral((1/10) * [1 2; 3 1])

[134]: true

[135]: prod_spec_d = is_productive_spectral([0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3])

[135]: true
```

Выводы

- Освоены пакеты Statistics, LinearAlgebra и BenchmarkTools.
- Выполнены поэлементные операции, нормы, разложения и решение СЛАУ.
- Оценена эффективность вычислений для матриц большого размера.
- Выполнены задания 4.4.1–4.4.4.